BankFlow

Banking Transaction Simulator with AI Support Ticket Routing and AIOps

1 Summary

BankFlow is a local banking simulator with a machine learning component for routing support requests. Users can record and reverse simulated transactions, classify support queries and review predicted labels. The project also implements a pipeline for model training, deployment and monitoring. The objective is to demonstrate consistent transaction handling and controlled updates to the model that serves the support inbox.

2 System design and implementation

The browser uses a configurable FastAPI URL. FastAPI manages the SQLite ledger and calls a separate MLflow model server over HTTP. This separation allows a model update to take place independently of the ledger. The DVC pipeline records experiments and model artifacts in local MLflow storage.

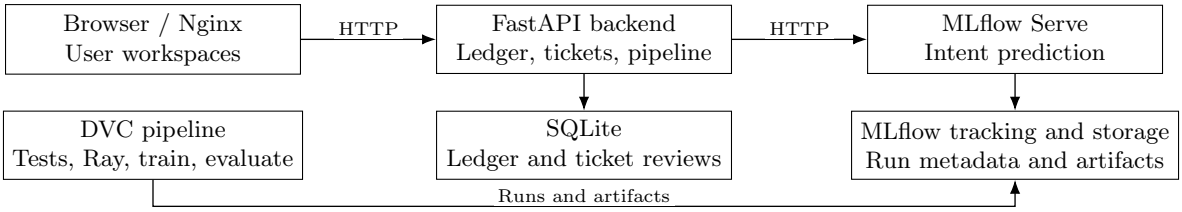


Figure 1: Application services, storage and the training pipeline.

Workspace	Implementation
Overview	Deposits, withdrawals, card transactions and linked reversals. Amounts are stored in integer paise. SQLite transactions enforce balance and credit limits. A matching retry returns the original transaction ID and current balances without creating a duplicate entry.
Support inbox	Shows the predicted intent, queue and confidence. Scores below 0.45 are flagged for review. The original prediction and the corrected label are recorded.
Operations	Starts a pipeline run and reports stage timings, failures, cache reuse, evaluation metrics and requested and active model versions.

Table 1: The three application workspaces.

Ledger consistency. A SQLite write transaction covers the retry lookup, balance checks and ledger insert. Reusing a request key with different data returns HTTP 409. A reversal records the opposite balance changes and links to the original transaction, which can be reversed only once.

Support routing. The model predicts the intent. Application rules assign it to the Top-ups, Cash & ATM, Transfers, Cards or Account support queue. A reviewer can change the effective queue by correcting the label. The ticket retains the original prediction and model run ID.

Endpoint	Request and result
`POST /api/transactions`	Takes the transaction type, amount in paise and request key. Returns updated balances and the transaction ID.
`POST /api/transactions/{id}/reverse`	Takes the original ID and a request key. Returns the reversal transaction ID and updated balances.
`POST /api/tickets`	Takes the query text. Returns intent, confidence, queue and model run ID. An invalid prediction or unavailable model returns HTTP 503 before a ticket is stored.
`PATCH /api/tickets/{id}`	Records the review label for an existing ticket.
`POST /api/pipeline/run`	Requests a background pipeline run. Returns HTTP 202 on acceptance or 409 if a run is active.

Table 2: Selected FastAPI endpoints. The low-level design specifies all 11 routes.

Evidence. Architecture, high-level design, low-level design and API specification, and Compose configuration.

3 Data preparation and model evaluation

BANKING77 contains short English queries across 77 banking intents. The bundled archive includes attribution, the CC BY 4.0 license and SHA-256 checksums. Ray Data validates the CSV batches and writes Parquet. The preparation stage removes duplicate training text and six train/test overlaps from the training split, checks for conflicting labels and retains all official test queries.

Split	Source rows	Prepared rows	Purpose
Training	10,003	9,993	Model selection and refitting
Official test	3,080	3,080	Held-out evaluation

Table 3: BANKING77 before and after preparation.

Preparation performance. The two splits were prepared in 3.63 seconds on the Ubuntu VM, equivalent to about 3,598 prepared rows per second. This wall-clock measurement includes Ray startup.

Model selection. The prepared training set is divided into stratified 80% fitting and 20% validation partitions with seed 42. A TF-IDF vectorizer with up to 10,000 unigram and bigram features is fitted on the fitting partition. Macro-F1 gives each intent equal weight when comparing the two classifiers.

Classifier	Validation accuracy	Validation macro-F1	Fit, prediction and scoring (s)
Multinomial Naive Bayes	81.19%	0.7865	0.043
Logistic regression	87.09%	**0.8707**	0.927

Table 4: Candidate comparison on the same validation split. Timings exclude data preparation.

Held-out result. After refitting, logistic regression achieved **88.47% accuracy** and **0.8851 macro-F1** on the 3,080 official test queries. The quality gate requires macro-F1 of at least 0.65. The baseline used zero reviewed training queries. The lowest per-intent F1 was 0.7126 for queries about a balance that did not update after a bank transfer. That class had 40 test examples.

Evidence. Preparation results, candidate results and per-class test results.

4 Pipeline and deployment

The MLproject entry point, Operations interface and monitoring worker use the same DVC pipeline. A lock serializes runs. DVC uses dependency hashes and parameters to decide which stages to execute.

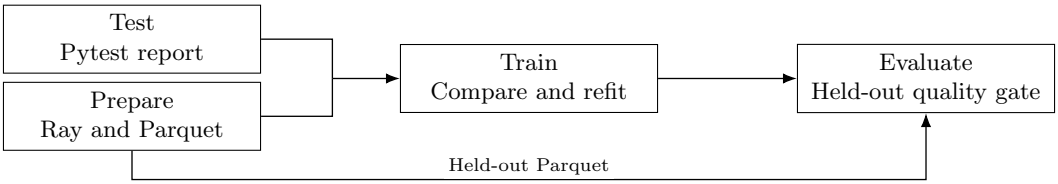


Figure 2: DVC stage dependencies. Training consumes the test report and prepared training data.

Experiment tracking. The parent MLflow run records evaluation and the deployment artifact. Its two child runs record the candidate results. The run artifacts include per-class metrics, the confusion matrix, feature weights, data hashes, package versions and reviewed feedback. The Git revision and source snapshot identify the executed code, and the MLflow run ID identifies the model artifact.

Deployment. A passing evaluation publishes the candidate run URI. The serving supervisor checks a prediction and its run ID before marking that version ready. Docker Compose runs the frontend, backend, trainer and model services, with a shared Python image for training and serving.

Condition	Behavior
Drift or scheduled review	Automatic retraining requires changed feedback and at least five eligible reviewed examples. A one-hour cooldown separates attempts.
Evaluation or load failure	A failed gate keeps the active model serving. A load failure can fall back to the last healthy version.
Operator rollback	Restores the previous healthy version. DVC reuse of an already approved candidate preserves the rollback.

Table 5: Retraining and model recovery behavior.

Evidence. DVC definition, MLproject execution and deployment and rollback log.

5 Monitoring and verification

Prometheus scrapes FastAPI metrics and queries the blackbox exporter. The Grafana dashboard has 16 panels for service health, requests, errors, latency, resource use, pipeline activity and model monitoring. The exporter probes the frontend, backend, model server, MLflow tracking server and Grafana.

Drift and review. After 50 queries are available, the monitor compares up to 200 recent queries with the training reference. It flags Jensen–Shannon divergence of at least 0.12 in query lengths or a rise of at least 15 percentage points in the share of unknown tokens. Reviewed agreement is calculated for the active model.

Check	Observed result
Ledger and API tests	Balance updates, retries and input validation passed. Pytest recorded 58 passes, zero failures, zero errors and zero skips.
Live application	Ledger, retries, reversals, real inference, label review and API contract checks passed.
Model lifecycle	The VM run submitted 52 synthetic tickets and saved five reviewed labels. Drift triggered retraining and deployment. A deliberately failing quality gate preserved the active version, and rollback restored the original version.
Health and alerts	All five service probes and Prometheus configuration and rule tests passed. A model outage fired an alert that cleared on recovery.
Frontend	Overview rendered at 320 by 480 pixels. The status message cleared when switching to Support inbox.

Table 6: Acceptance results on the Ubuntu ARM64 VM.

Evidence. Validation plan, pytest report and service probes.

6 Scope and limitations

BankFlow uses simulated balances and runs on a single VM. The classifier is evaluated on short English queries from the 77 BANKING77 intents. Its accuracy and macro-F1 describe that test set. The drift monitor measures changes in input characteristics, while reviewed agreement uses the human labels available for the active model. The preparation timing describes the recorded local workload.